

Data Analysis with Python

Week 4: Tidy Data, Reshaping, and Group-wise Operations

Justinas Grigaitis, 2026-10-08

Outline for next weeks

Data Analysis with Python

- Week 1-2: Python fundamentals
- Week 3: Intro to data analysis, cleaning, and Pandas
- Week 4: Data wrangling and analysis (focus on reshaping and agg/transform)
- Week 5: quiz, guest talk on AI research/engineering, and data visualization
 - Python fundamentals quiz (20% of grade)
- Week 6: group project idea pitches and sample project walkthrough

Plan for today

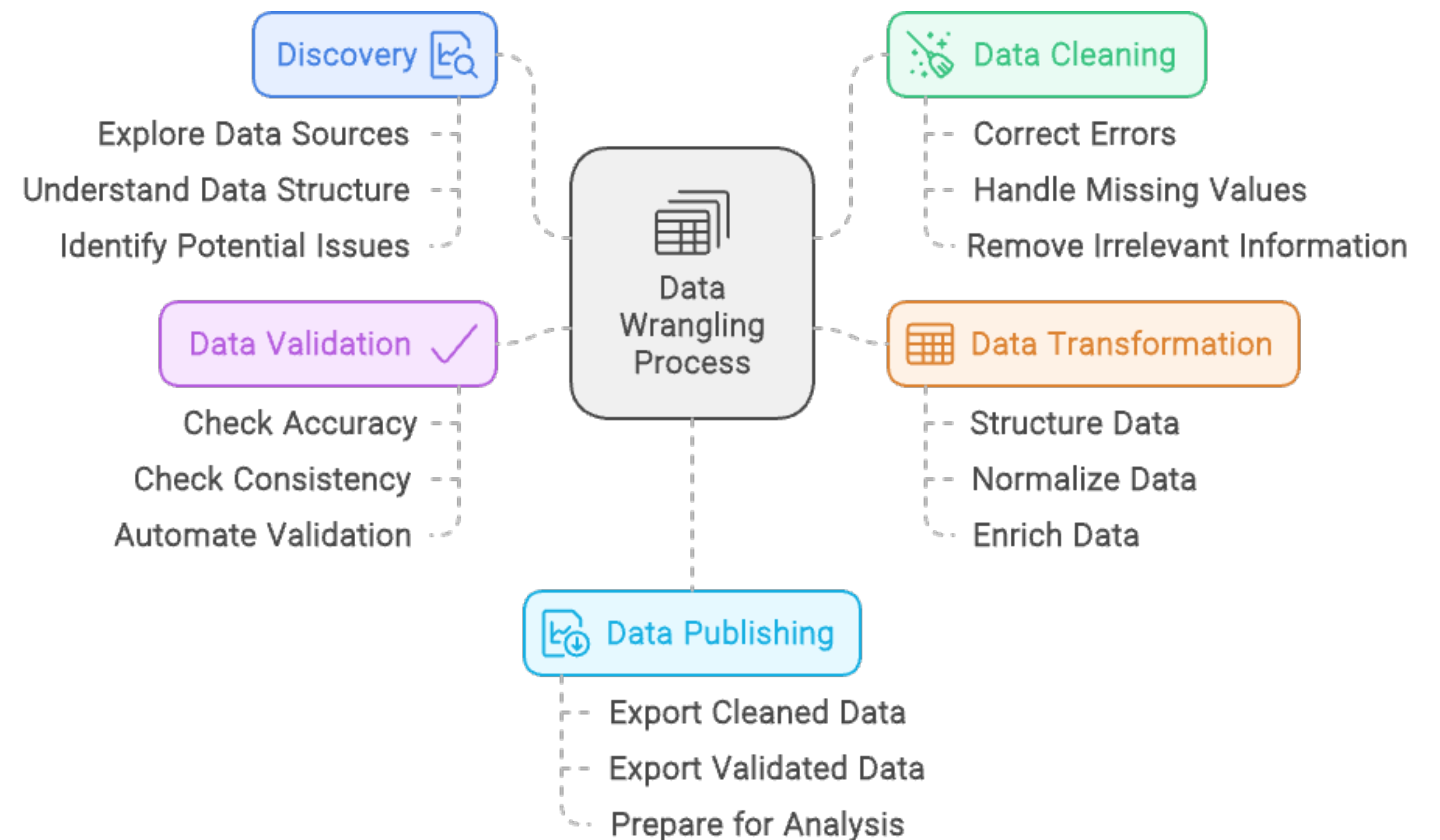
- Feedback (~10 min)
- Fundamentals recap, coding (~30 min)
- Presentation on data reshaping (~15 min)
- Break (~10 min)
- Activity (~15 min)
- Solutions on Streamlit (~25 min)

Data wrangling and analysis

Crash course on data wrangling

Plan -> clean -> validate -> output

- Discovery - diagnostics and plan
- Structuring - combine and reshape
- Cleaning - edit variables
- Enriching - add new variables
- Validating - sanity checks
- Publishing - output
- And then analyse!

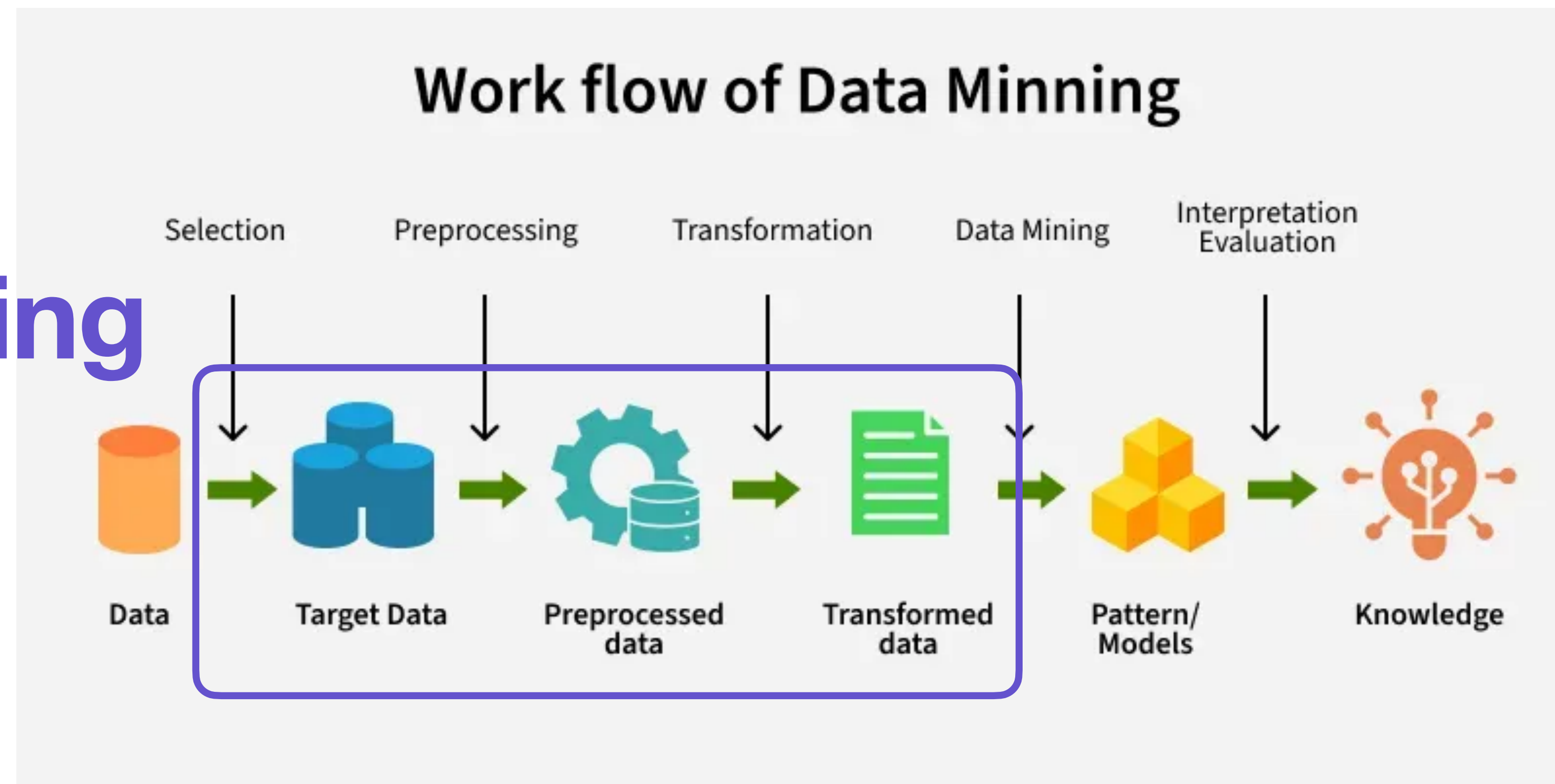


Sources:

<https://www.alteryx.com/glossary/data-wrangling>

<https://www.datacamp.com/blog/what-is-data-wrangling>

Wrangling



Selecting

- Remove
- Variables/rows

Cleaning

- Edit/add
- Variables

Merging

- Append
- Join

Reshaping

- Aggregate
- Melt and Pivot

Tidy data

Happy datasets are all alike...

Source: <https://vita.had.co.nz/papers/tidy-data.pdf>

1. Each variable forms a column.
2. Each observation forms a row.
3. Each type of observational unit forms a table.

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	216766	1280428583

variables

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	216766	1280428583

observations

country	year	cases	population
Afghanistan	999	745	19987071
Afghanistan	000	2666	20595360
Brazil	999	37737	172006362
Brazil	000	80488	174504898
China	999	212258	1272915272
China	000	216766	1280428583

values

Tidy tools in pandas

Cheat Sheet is all you need

Data Wrangling with pandas Cheat Sheet

<http://pandas.pydata.org>

Tidy Data – A foundation for wrangling in pandas

In a tidy data set:

Each **variable** is saved in its own **column** & Each **observation** is saved in its own **row**

Tidy data complements pandas's **vectorized operations**. pandas will automatically preserve observations as you manipulate variables. No other format works as intuitively with pandas.

Syntax – Creating DataFrames

	a	b	c
1	4	7	10
2	5	8	11
3	6	9	12

```
df = pd.DataFrame(  
    {"a" : [4 ,5, 6],  
     "b" : [7, 8, 9],  
     "c" : [10, 11, 12]},  
    index = [1, 2, 3])  
Specify values for each column.
```

```
df = pd.DataFrame(  
    [[4, 7, 10],  
     [5, 8, 11],  
     [6, 9, 12]]
```

Reshaping Data – Change the layout of a data set

`pd.melt(df)`
Gather columns into rows.

`df.pivot(columns='var', values='val')`
Spread rows into columns.

```
df.sort_values('mpg')  
Order rows by values of a column (low to high).
```

```
df.sort_values('mpg',ascending=False)  
Order rows by values of a column (high to low).
```

```
df.rename(columns = {'y':'year'})  
Rename the columns of a DataFrame
```

```
df.sort_index()  
Sort the index of a DataFrame
```

```
df.reset_index()  
Reset index of DataFrame to row numbers, moving  
index to columns.
```

<https://pandas.pydata.org/Pandas Cheat Sheet.pdf>

https://pandas.pydata.org/docs/user_guide/index.html

Data cleaning - recap

Data cleaning

Make dataset usable

- Explore: `df.describe()` and `df.info()`, rename and drop variables
- Type casting: convert to appropriate variable type
 - `pd.to_numeric()`, `pd.to_datetime()`, `Series.astype(str)`
- Change variable values or add new variables with:
 - Numeric operations: basic math
 - String operations: slice, replace, extract parts (regex), split, dummy variables
 - Date operations: extract hours, days, weeks, months, years
- Filtering: always based on condition, never based on position
 - `df.loc[]`, `np.where()`, `np.select()`
 - Missing values and duplicates

Import/Export

Make use of base Python

- Choose the right pandas function based on file format (.csv, .xlsx, .dta, etc.)
 - Bonus: specify variable types upon importing (e.g., if you know which variables are date/numeric)
- Use *os* or *pathlib* libraries to read file names and create folders
- Use loops, dictionaries, lists when importing and exporting for automation

Reshaping

Main source: https://pandas.pydata.org/docs/user_guide/reshaping.html

Reshaping

Long and wide formats

Source: [TowardsDataScience.com](https://towardsdatascience.com)

- Long format: column names = variable names
- Wide format: column names = values

Long Format

	Series ID	Item	Year Month	Avg. Price (\$)
0	APU0000702111	Bread, white per lb	2020 Jan	1.351
1	APU0000702111	Bread, white per lb	2020 Feb	1.375
2	APU0000702111	Bread, white per lb	2020 Mar	1.374
3	APU0000702111	Bread, white per lb	2020 Apr	1.406
4	APU0000702111	Bread, white per lb	2020 May	1.412
...	...	...	...	...
134	APU0000709112	Milk, Whole per gal	2021 Dec	3.743
135	APU0000709112	Milk, Whole per gal	2022 Jan	3.787
136	APU0000709112	Milk, Whole per gal	2022 Feb	3.875
137	APU0000709112	Milk, Whole per gal	2022 Mar	3.917
138	APU0000709112	Milk, Whole per gal	2022 Apr	4.012

Wide Format

	Series ID	Item	Year Month	2020 Jan	2020 Feb	2020 Mar	2020 Apr	2020 May	2020 Jun	2020 Jul	2020 Aug	2020 Sep	2020 Oct	...	2021 Jul	2021 Aug	2021 Sep	2021 Oct
	APU0000702111	Bread, white per lb		1.351	1.375	1.374	1.406	1.412	1.474	1.485	1.495	1.492	1.503	...	1.491	1.467	1.580	1.526
	APU0000703112	Ground beef, 100% per lb		3.886	3.865	3.881	4.052	4.461	4.737	4.264	4.177	4.076	4.008	...	4.388	4.468	4.504	4.719
	APU0000706111	Chicken, whole per lb		1.409	1.362	1.400	1.571	NaN	1.747	1.712	1.609	1.542	1.580	...	1.435	1.472	1.504	1.523
	APU0000708111	Eggs, large per doz		1.461	1.449	1.525	2.019	1.640	1.554	1.401	1.328	1.353	1.408	...	1.642	1.709	1.835	1.821
	APU0000709112	Milk, Whole per gal		3.253	3.196	3.248	3.267	3.210	3.198	3.255	3.406	3.448	3.380	...	3.627	3.560	3.585	3.663

Reshaping

Long and wide formats

See: [data_layout_examples.xlsx](#)

- Long data ~ tidy data
 - Key (index) - value pairs: observations are unique by index (“key” variables)
 - Starting point for further analysis
 - Easier for calculating group summary statistics
- Wide data
 - Emphasis on comparisons between groups (e.g., years, countries)
 - Used in final deliverables (e.g., for presentation)
 - Used for plotting in Excel (but not in Python!)

Reshaping

Vocabulary overview

- Reshaping = Transposing
- Long to wide:
 - Python: `pivot`, `pivot_table`, `unstack`; `get_dummies`
 - R: `pivot_wider` (previously `spread`), `unstack`
- Wide to long:
 - Python: `melt`, `stack`, `wide_to_long`; use `reset_index()` afterwards; `from_dummies`
 - R: `pivot_longer` (previously `gather`), `stack`, `melt`
- Stata: `reshape long` / `reshape wide`

Reshaping

Wide to long - melt()

df3

	first	last	height	weight
0	John	Doe	5.5	130
1	Mary	Bo	6.0	150



df3.melt(id_vars=['first', 'last'])

	first	last	variable	value
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130
3	Mary	Bo	weight	150

id_vars: what stays unchanged

value_vars: variable names of columns to be transposed

var_name / *value_name*: name of the new group indicator and value variables (default “variable” and “value”)

Use *reset_index()* afterwards

Reshaping

Wide to long - melt()

Source: [blog post](#)

	student	school	english	math	physics
0	Andy	Z	10	20	30
1	Bernie	Y	100	200	300
2	Cindy	Z	1000	2000	3000
3	Deb	Y	10000	20000	30000

```
pd.melt(frame=df_wide,  
        id_vars="student",  
        var_name="cLaSs",  
        value_name="gRaDe")
```



	student	cLaSs	gRaDe
0	Andy	school	Z
1	Bernie	school	Y
2	Cindy	school	Z
3	Deb	school	Y
4	Andy	english	10
5	Bernie	english	100
6	Cindy	english	1000
7	Deb	english	10000
8	Andy	math	20
9	Bernie	math	200
10	Cindy	math	2000
11	Deb	math	20000
12	Andy	physics	30
13	Bernie	physics	300
14	Cindy	physics	3000
15	Deb	physics	30000

Reshaping

Long to wide - pivot()

index: what stays unchanged

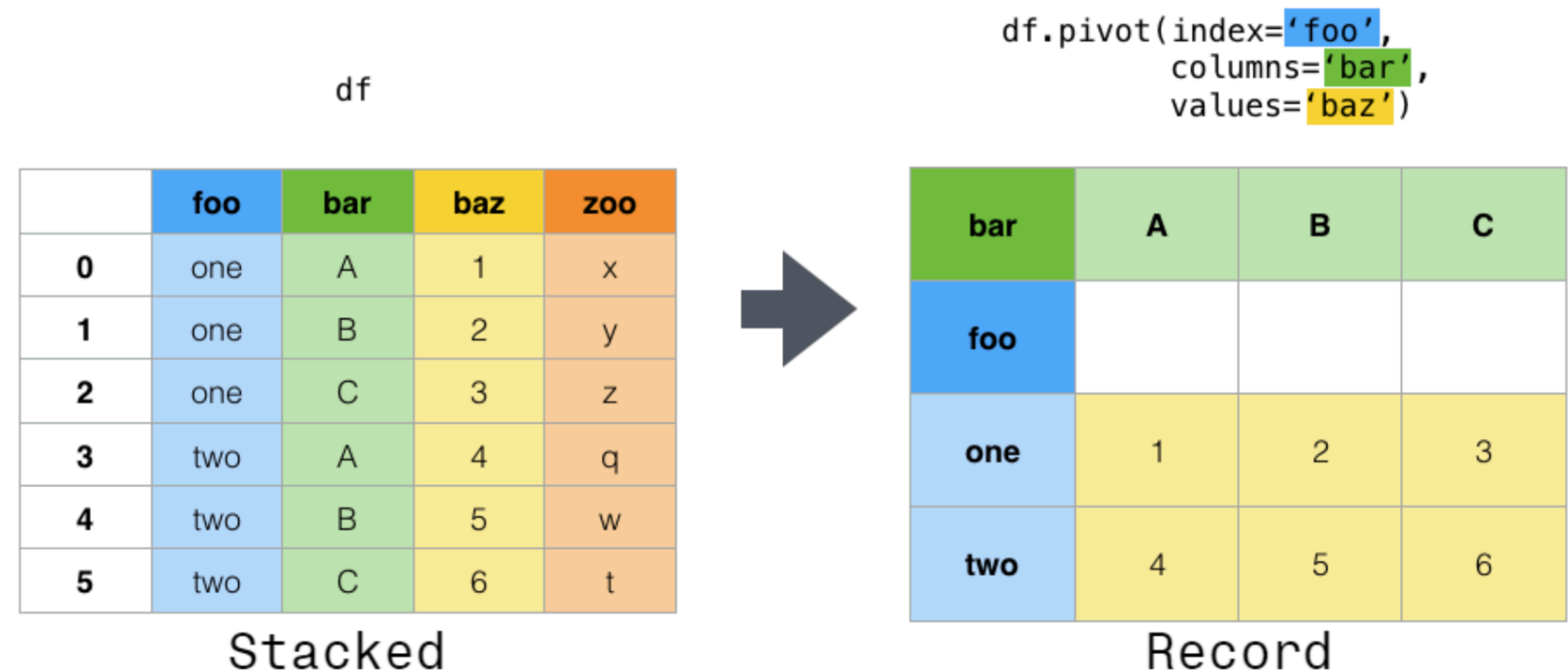
columns: variable name of the (group indicator) column to be transposed

values: variable name of the value column to be used

Note that values have to be unique by index-columns (otherwise use pivot_table)

It's possible to transpose multiple columns and multiple values at once, but it gets quite complicated (-> multi-index dataframe).

I would recommend sticking to one column-value pair when pivoting, and if needed, creating a new combined group variable column prior to pivoting.



Reshaping

Long to wide - pivot_table() with aggregation function

Source: [blog post](#)

Pivot 1

student	school	class	grade
0	Andy	Z	english 10
1	Bernie	Y	english 100
2	Cindy	Z	english 1000
3	Deb	Y	english 10000
4	Andy	Z	math 20
5	Bernie	Y	math 200
6	Cindy	Z	math 2000
7	Deb	Y	math 20000
8	Andy	Z	physics 30
9	Bernie	Y	physics 300
10	Cindy	Z	physics 3000
11	Deb	Y	physics 30000

```
df_long.pivot_table(index=["student", "school"],  
                    columns='class',  
                    values='grade').reset_index()
```

class	student	school	english	math	physics
0	Andy	Z	10	20	30
1	Bernie	Y	100	200	300
2	Cindy	Z	1000	2000	3000
3	Deb	Y	10000	20000	30000

Pivot 2

```
df_long.pivot_table(index=["student", "school"],  
                    columns='class',  
                    values='grade',  
                    margins=True,  
                    aggfunc='sum').reset_index()
```

class	student	school	english	math	physics	All
0	Andy	Z	10	20	30	60
1	Bernie	Y	100	200	300	600
2	Cindy	Z	1000	2000	3000	6000
3	Deb	Y	10000	20000	30000	60000
4	All		11110	22220	33330	66660

Pivot 3

student	school	class	grade
0	Andy	Z	english 10
1	Bernie	Y	english 100
2	Cindy	Z	english 1000
3	Deb	Y	english 10000
4	Andy	Z	math 20
5	Bernie	Y	math 200
6	Cindy	Z	math 2000
7	Deb	Y	math 20000
8	Andy	Z	physics 30
9	Bernie	Y	physics 300
10	Cindy	Z	physics 3000
11	Deb	Y	physics 30000

```
df_long.pivot_table(index=["student", "school"],  
                    # default aggfunc='mean'
```

student	school	grade
Andy	Z	20
Bernie	Y	200
Cindy	Z	2000
Deb	Y	20000

$(10+20+30)/3$
 $(100+200+300)/3$
 $(1000+2000+3000)/3$
 $(10000+20000+30000)/3$

Pivot 4

```
df_long.pivot_table(index="student",  
                    columns=['school', 'class'],  
                    values='grade',  
                    fill_value=-5)
```

student	Y			Z			
	class	english	math	physics	english	math	physics
Andy		-5	-5	-5	10	20	30
Bernie		100	200	300	-5	-5	-5
Cindy		-5	-5	-5	1000	2000	3000
Deb		10000	20000	30000	-5	-5	-5

Aggregation and Transformation

Main source: https://pandas.pydata.org/docs/user_guide/groupby.html

Aggregation and Transformation

Collapsing vs preserving the dataset

- Aggregation = summarization= collapsing
 - Examples: sum and average, count unique values, list unique values
 - Functions: `df["y"].agg("mean")`, `df["y"].nunique()`
 - In R: `summarise()`, in Stata: `collapse()`
- Transformation = preserving shape
 - Examples: cumulative sum, running average, total sum
 - Function: `df["y"].transform("cumsum")`, `df["y"].rolling(window=7).mean()`
 - In R: `mutate()`, in Stata: `by/bysort :... egen/gen`

Aggregation and Transformation

Everything can be done by group: split-apply-combine

Source: [blog post, https://pandas.pydata.org/docs/user_guide/groupby.html](https://pandas.pydata.org/docs/user_guide/groupby.html)

- `df.groupby()` creates a GroupBy object, which can be used with aggregation, transformation, and filtration functions
- IMPORTANT: always set `df.groupby(dropna=False)`

```
df.groupby(  
    ['nationality', 'degree']  
) .agg(  
    mean_salary = ('salary', 'mean'),  
    min_age     = ('age', 'min'),  
    max_age     = ('age', 'max')  
) .reset_index()
```

Group by these column

Renamed output columns

Operation on columns

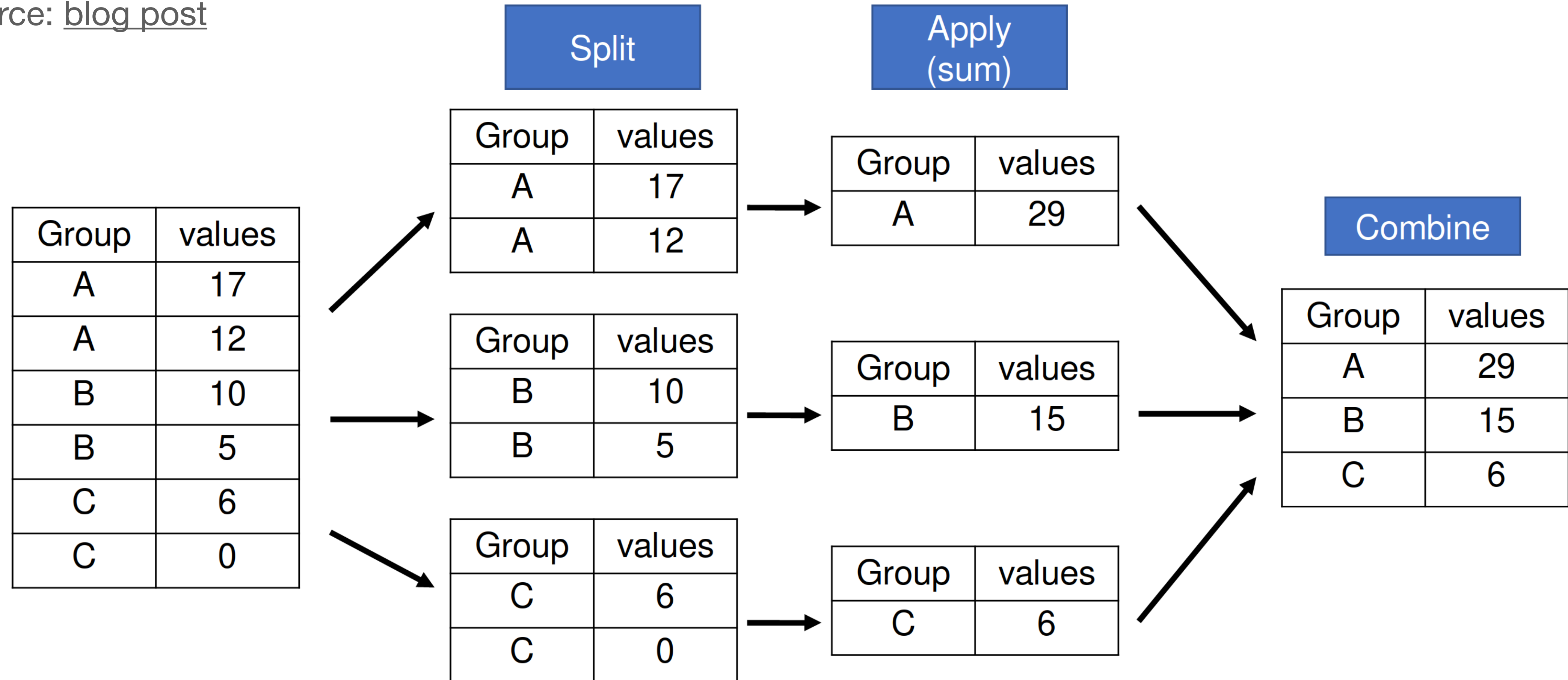
Selected columns



Aggregation

Split-apply-combine with `df.groupby().agg()`

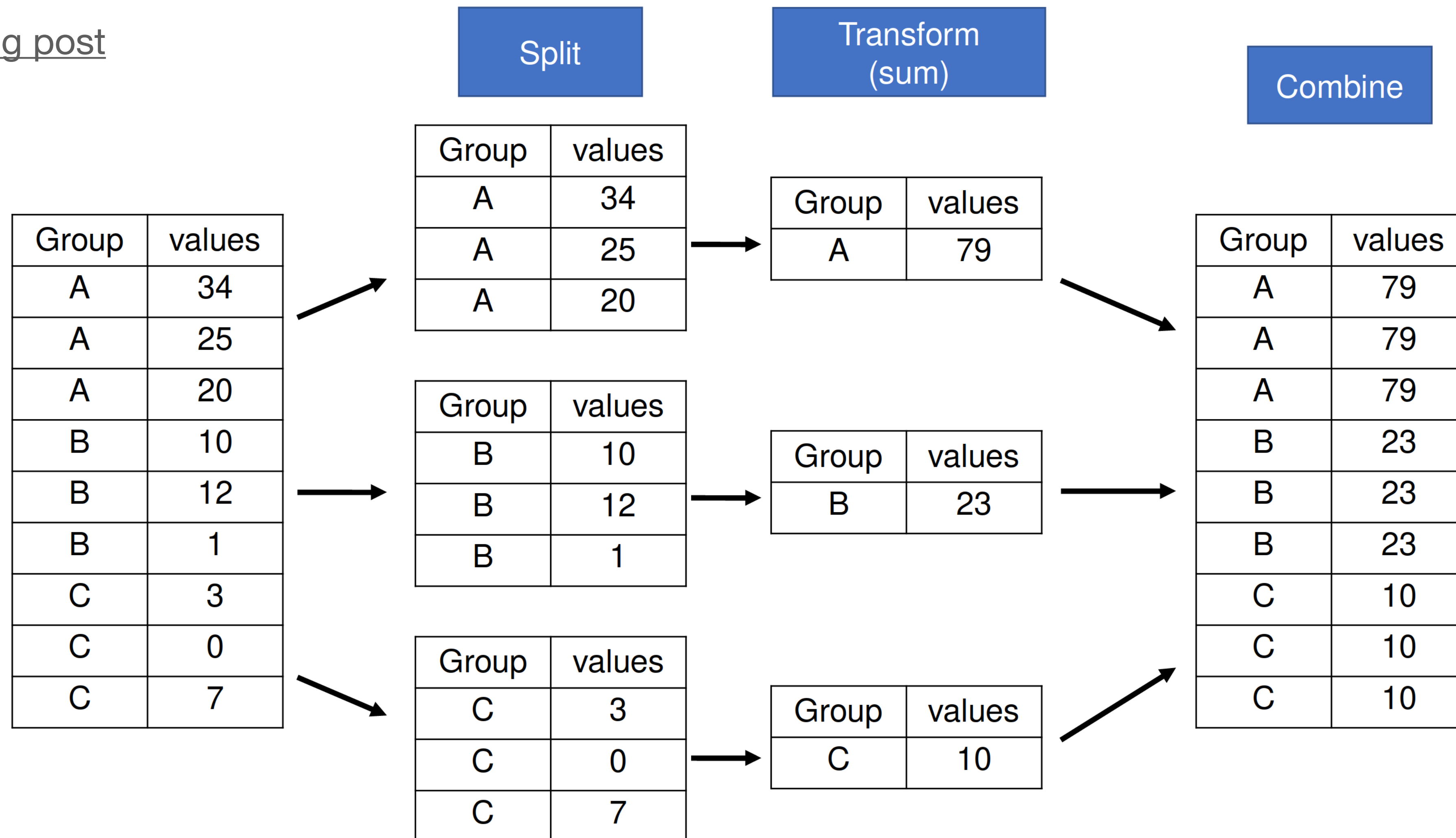
Source: [blog post](#)



Transformation

Split-apply-combine with `df.groupby().transform()`

Source: [blog post](#)



Practice Activity

Solutions Pseudocode and Streamlit

Python for frontend

Web apps with Python

Easy way to the outside world

- Python is primarily a back-end oriented language
- New libraries adapted it for front-end: Django, Flask, FastAPI, Plotly Dash
- More recently, a wave of very user-friendly tools: Streamlit, Gradio, Taipy, etc.
- Most importantly, a change in mindset in the age of GenAI
- Tutorials, videos: [fast prototyping](#), [data reports vs apps](#), <https://30days-tmp.streamlit.app/>

```
import streamlit as st
import pandas as pd
import numpy as np

st.write("Streamlit supports a wide range of data visualizations, i

all_users = ["Alice", "Bob", "Charly"]
with st.container(border=True):
    users = st.multiselect("Users", all_users, default=all_users)
    rolling_average = st.toggle("Rolling average")

np.random.seed(42)
data = pd.DataFrame(np.random.randn(20, len(users)), columns=users)
if rolling_average:
    data = data.rolling(7).mean().dropna()

tab1, tab2 = st.tabs(["Chart", "Dataframe"])
tab1.line_chart(data, height=250)
tab2.dataframe(data, height=250, use_container_width=True)
```

Share

Streamlit supports a wide range of data visualizations, including [Plotly](#), [Altair](#), and [Bokeh charts](#). 🇮🇹 And with over 20 input widgets, you can easily make your data interactive!

Users

Alice ×

Bob ×

Charly ×

☐ Rolling average

Chart Dataframe



Streamlit for data analysis

Reproducibility, interactivity, and communication

- Streamlit as the new IDE?
 - Code must run -> reproducibility
 - Interactive widgets -> flexibility
 - Richer layout -> visual communication
 - Multi-page structure -> clear roadmap
- Examples of my projects
 - Data analysis from EMP / M2P
 - Research paper draft
 - Theory visualization

Homework

Homework (~5 hours)

Prep for quiz and pandas

- Fundamentals practice - all from weeks 1-2 (3 hr)
- Basics of data wrangling in pandas (1 hr)
- Intro to Streamlit (1 hr)
 - Learn how to open Streamlit app via terminal (*streamlit run app.py*)
 - On your page, write text (*st.markdown()*) and display data (*st.dataframe()*)
 - Add a filter (*st.multiselect()* or *st.radio()*) and limit your dataset based on string variables (e.g., movies vs series)